

Veri Tabanı Final

Exported on: 11.01.2026 14:33:13

SQL Server İleri Düzey Veritabanı Kavramları: Brifing Belgesi

Yönetici Özeti

Bu brifing belgesi, Microsoft SQL Server Veritabanı Yönetim Sistemleri'nin ileri düzey yeteneklerini kapsayan temel konuları sentezlemektedir. Belge, Balıkesir Üniversitesi Bilgisayar Mühendisliği Bölümü tarafından hazırlanan ders notlarına dayanarak, ilişkisel veritabanlarında veri manipülasyonu, sorgulama ve otomasyon için kritik olan altı ana konuyu derinlemesine incelemektedir: JOIN , VIEW , İç İç SELECT Sorguları (Subqueries), FUNCTION , Stored Procedure ve TRIGGER . Her bir bölüm, kavramların teorik altyapısını, pratik kullanım senaryolarını, sözdizimi kurallarını ve performans üzerindeki etkilerini detaylı örneklerle açıklamaktadır.

• **JOIN İşlemleri:** İki veya daha fazla tabloyu, aralarındaki ilişkisel sütunlar üzerinden birleştirerek anlamlı veri setleri oluşturma yöntemlerini inceler. INNER JOIN , OUTER JOIN (LEFT, RIGHT, FULL) ve CROSS JOIN gibi temel birleştirme türlerinin mantığı ve uygulama farklılıkları örneklerle ortaya konulmuştur.

• **VIEW Nesneleri (Görünümler):** Fiziksel veri içermeyen, saklanmış SELECT sorguları olarak işlev gören sanal tabloları ele alır. Karmaşık sorguları basitleştirme, veri güvenliğini artırma ve performansı iyileştirme gibi amaçları detaylandırılır. CREATE , ALTER , DROP komutlarıyla yönetimleri ve kısıtlamaları belirtilmiştir.

• **İç İç SELECT Sorgular (Subqueries):** Bir sorgunun sonucunu başka bir sorgu için filtreleme veya veri kaynağı olarak kullanma tekniğidir. WHERE , HAVING veya FROM ifadeleri içinde nasıl konumlandırıldıkları, tek satır ve çok satır döndüren alt sorgular arasındaki farklar ve kullanılan operatörler (IN , ANY , ALL) açıklanmıştır.

• **Kullanıcı Tanımlı Fonksiyonlar (Functions):** Belirli bir görevi yerine getiren ve geriye bir değer (sabit, sorgu sonucu veya tablo) döndüren yeniden kullanılabilir SQL kod bloklarıdır. Stored Procedure 'lerden temel farkları, oluşturma türleri ve sorgu içinde doğrudan kullanılabilme avantajları vurgulanmıştır.

• **Saklı Yordamlar (Stored Procedures):** Veritabanı sunucusunda derlenmiş olarak saklanan ve EXECUTE komutuyla çalıştırılan SQL ifadeleridir. Performans, güvenlik, ağ trafiğini azaltma gibi avantajları ile öne çıkarlar. Parametre alabilme, CRUD işlemlerini gerçekleştirebilme ve Transaction yönetimi gibi yetenekleri incelenmiştir.

• **Tetikleyiciler (Triggers):** Bir tabloda INSERT , UPDATE veya DELETE gibi bir DML işlemi gerçekleştiğinde otomatik olarak çalışan özel saklı yordamlardır. Veri bütünlüğünü sağlama, denetim kaydı tutma ve karmaşık iş kurallarını uygulama gibi amaçlarla kullanılırlar. AFTER ve INSTEAD OF türleri, inserted ve deleted sanal tablolarının kullanımı ile işlem kontrolü (ROLLBACK) konuları ele alınmıştır.

Bölüm 1: Tablolar Arası İlişki Kurma (JOIN Kullanımı)

JOIN ifadeleri, iki veya daha fazla tabloyu ya da görünümü (view) aralarındaki ilişkisel sütunlara dayanarak birleştirmek için kullanılır. Bu birleştirme, SELECT , UPDATE , DELETE gibi veri işleme komutlarının FROM yan tümcesinde gerçekleştirilir. Temel amaç, farklı tablolarda depolanan verileri tek bir sorgu sonucunda bir araya getirmektir.

Genel Sözdizimi: FROM ilk_tablo join_turu ikinci_tablo ON (join_kosulu)

• **join_turu:** Hangi birleştirme yönteminin (INNER , OUTER , CROSS) kullanılacağını belirtir.

• **join_kosulu:** Tabloların hangi sütunlar üzerinden ve hangi koşulla eşleştirileceğini tanımlar.

1.1. INNER JOIN (İç Birleştirici)

INNER JOIN , SQL Server'da varsayılan birleştirme türüdür ve iki tabloda da ON koşulunu sağlayan, yani ortak olan satırları getirir. Bire bir ilişki veya her iki tabloda da karşılığı olan kayıtları listelemek için kullanılır.

• Özellikleri:

- Yalnızca her iki tablodaki eşleşen satırları döndürür.
- Genellikle = veya <> operatörleri ile kullanılır.
- Sözdiziminde INNER JOIN yerine sadece JOIN yazmak yeterlidir.

• Örnek Kullanım: Danışmanı olan öğrencileri listelemek.

• Bu sorgu, OgrenciDanisman tablosunda kaydı bulunan, yani bir danışmana atanmış öğrencileri Ogrenci tablosundan getirir.

1.2. OUTER JOIN (Dış Birleştirici)

OUTER JOIN , INNER JOIN 'den farklı olarak, birleştirilen tablolardan birindeki tüm kayıtları, diğer tablodaki eşleşen kayıtlarla birlikte getirir. Eşleşme bulunmayan durumlarda, diğer tablodan gelen sütunlar için NULL değeri döndürülür. Genellikle bir tabloda olup diğerinde olmayan kayıtları tespit etmek için kullanılır. Üç ana türü vardır:

1.2.1. LEFT JOIN (veya LEFT OUTER JOIN)

İfadenin sol tarafındaki (ilk belirtilen) tablonun **tüm satırlarını** ve sağ taraftaki tablodan yalnızca eşleşen satırları getirir. Sağ tabloda eşleşme yoksa, o tablodan gelen sütunlar NULL olur.

• Örnek Kullanım: Henüz bir danışmanı olmayan öğrencileri listelemek.

• Bu sorgu, tüm öğrencileri listeler ancak OgrenciDanisman tablosunda karşılığı olmayan (DanismanId sütunu NULL olan) öğrenciler, danışmanı olmayan öğrencilerdir.

1.2.2. RIGHT JOIN (veya RIGHT OUTER JOIN)

İfadenin sağ tarafındaki (ikinci belirtilen) tablonun **tüm satırlarını** ve sol taraftaki tablodan yalnızca eşleşen satırları getirir. Sol tabloda eşleşme yoksa, o tablodan gelen sütunlar NULL olur.

• Örnek Kullanım: Henüz bir öğrencisi olmayan danışmanları listelemek.

• Bu sorgu, Danisman tablosundaki tüm danışmanları getirir ve OgrenciDanisman tablosunda karşılığı olmayanları filtreleyerek hiçbir öğrenciye atanmamış danışmanları bulur.

1.2.3. FULL JOIN (veya FULL OUTER JOIN)

Hem sol hem de sağ tablodaki **tüm satırları** listeler. Eşleşme olan satırları birleştirir, eşleşme olmayan satırlar için ise karşı tablodan gelen sütunlara NULL değerini atar.

• Örnek Kullanım: Tüm öğrencileri ve tüm danışmanları, aralarındaki eşleşmelerle birlikte (veya eşleşme olmasa bile) listelemek.

• Bu sorgunun sonucunda danışmanı olmayan öğrenciler, öğrencisi olmayan danışmanlar ve eşleşen öğrenci-danışman kayıtları tek bir sonuç setinde yer alır.

1.3. CROSS JOIN (Çapraz Birleştirici)

Kartezyen çarpım (cartesian product) olarak da bilinen CROSS JOIN , iki tablo arasında mümkün olan tüm satır kombinasyonlarını oluşturur. ON koşulu belirtilmez. Sol tablodaki her bir satırı, sağ tablodaki tüm satırlarla eşleştirir.

- **Örnek Kullanım:** Her bir öğrenci için tüm olası danışmanları listelemek.
- Eğer Öğrenci tablosunda 4, Danışman tablosunda 4 kayıt varsa, sonuç olarak $4 * 4 = 16$ satırlık bir tablo döner.

1.4. DELETE ve UPDATE İşlemlerinde JOIN Kullanımı

JOIN ifadeleri yalnızca veri çekmek için değil, aynı zamanda koşullu veri silme ve güncelleme işlemleri için de kullanılabilir. Bu, başka bir tablodaki verilere dayanarak bir tablodaki kayıtları değiştirmeyi kolaylaştırır.

- **Örnek Kullanım:** Danışmanı olmayan öğrencileri silmek.
- Bu komut, LEFT JOIN ile danışmanı olmayan öğrencileri tespit eder ve Öğrenci tablosundan bu kayıtları siler.

Bölüm 2: Sanal Tablolar Oluşturma (VIEW Nesneleri)

VIEW (Görünüm), içerisinde SELECT sorgularını saklayan sanal bir veritabanı nesnesidir. Fiziksel olarak veri depolamaz; bunun yerine, temel aldığı bir veya daha fazla tablodan verileri dinamik olarak çeker. Her çağrıldığında, tanımlı olduğu sorguyu yeniden çalıştırır.

2.1. VIEW Kavramı ve Amaçları

- **Sanal Tablolar:** VIEW 'ler, gerçek tablolar gibi sorgulanabilir ancak kendilerine ait fiziksel verileri yoktur.
- **Karmaşıklığı Azaltma:** Karmaşık JOIN veya UNION içeren sorguları tek bir VIEW altında toplayarak verilere daha kolay ve hızlı erişim sağlar.
- **Güvenlik:** Kullanıcılara bir tablonun tamamı yerine sadece belirli satır ve sütunlarını görme yetkisi vererek veri erişimini kısıtlar.
- **Veri Tutarlılığı:** Verilerin önbelleğe alınmasını ve daha hızlı erişilmesini sağlayarak performansı artırabilir.

2.2. VIEW Kullanımının Kısıtlamaları

- İsimsiz bir sütun kullanılamaz; her sütunun bir adı olmalıdır.
- Parametre alamazlar (Stored Procedure 'ler gibi).
- Tanımlarında INSERT , UPDATE , DELETE gibi DML komutları kullanılamaz; sadece SELECT ifadeleri bulunabilir.
- ORDER BY ifadesi, TOP veya OFFSET-FETCH ile birlikte kullanılmadığı sürece VIEW tanımında kullanılamaz.

2.3. VIEW Yönetimi

VIEW Oluşturma (CREATE VIEW)

```
CREATE VIEW VW_OğrenciDetay
AS
SELECT
    O.ID,
    O.Ad,
    O.Soyad,
    C.Cinsiyet,
    S.Sehir
```

```
FROM TblOgrenci O
JOIN TblSehir S ON S.ID = O.SehirID
JOIN TblCinsiyet C ON C.ID = O.CinsiyetID;
```

Bu VIEW oluşturulduktan sonra, basit bir tablo gibi sorgulanabilir: `SELECT * FROM VW_OgrenciDetay;`

VIEW Güncelleme (ALTER VIEW)

Mevcut bir VIEW 'in tanımını değiştirmek için `ALTER VIEW` komutu kullanılır.

```
ALTER VIEW VW_OgrenciDetay AS ... -- Yeni SELECT sorgusu
```

VIEW Silme (DROP VIEW)

Bir VIEW 'i veritabanından kaldırmak için `DROP VIEW` komutu kullanılır.

```
DROP VIEW IF EXISTS VW_OgrenciDetay;
```

Güncellenebilir VIEW'ler

Tek bir tablodan türetilmiş ve belirli koşulları (agregasyon fonksiyonları, `GROUP BY`, `DISTINCT` vb. içermeyen) sağlayan VIEW 'ler üzerinden `INSERT`, `UPDATE`, `DELETE` işlemleri yapılabilir. Bu işlemler doğrudan temel tabloyu etkiler.

Bölüm 3: İç İç SELECT Sorguları (Subqueries)

İç içe sorgular (Subqueries), bir `SELECT`, `INSERT`, `UPDATE` veya `DELETE` sorgusunun `WHERE`, `HAVING` ya da `FROM` yan tümcesi içine yerleştirilmiş başka bir `SELECT` sorgusudur. Ana sorgu (`outer query`), iç sorgunun (`subquery` veya `inner query`) döndürdüğü sonucu kullanarak kendi işlemini gerçekleştirir.

3.1. Kullanım Mantığı ve Kuralları

- **Değer Aktarımı:** İç sorgu, bir üstteki sorguya değer veya değerler kümesi gönderir.
- **Parantez Kullanımı:** Alt sorgular daima parantez () içinde yazılmalıdır.
- **ORDER BY Kısıtlaması:** Alt sorgularda `ORDER BY` komutu kullanılamaz (ancak `TOP` gibi ifadelerle birlikte istisnai durumlar olabilir). Sıralama, ana sorguda yapılır.
- **Konulandırma:** Genellikle `WHERE` ve `HAVING` yan tümcelerinde filtreleme için veya `FROM` yan tümcesinde geçici bir tablo oluşturmak için kullanılırlar.

3.2. Tek Satır ve Çok Satır Döndüren Alt Sorgular

Alt sorguların döndürdüğü sonuç setinin satır sayısına göre kullanılan karşılaştırma operatörleri değişir.

Tek Satır Döndüren Alt Sorgular

Bu sorgular, ana sorguya yalnızca tek bir değer (tek satır, tek sütun) döndürür. Bu durumda standart karşılaştırma operatörleri kullanılır.

- **Operatörler:** `=`, `<`, `>`, `<=`, `>=`, `<>`
- **Örnek:** Ortalama IMDB puanından daha yüksek puana sahip filmleri listelemek.

Çok Satır Döndüren Alt Sorgular

Bu sorgular, ana sorguya birden fazla değer içeren bir liste (çok satır, tek sütun) döndürür. Bu değer listesiyle karşılaştırma yapmak için özel operatörler kullanılır.

• **Operatörler:** IN , NOT IN , ANY , ALL

• **IN Operatörü:** Değerin, alt sorgunun döndürdüğü listedeki herhangi bir değerle eşleşip eşleşmediğini kontrol eder.

• **Örnek:** "Dram" türündeki filmleri listelemek.

Bölüm 4: Kullanıcı Tanımlı Fonksiyonlar (FUNCTION Kullanımı)

Kullanıcı tanımlı fonksiyonlar (User-Defined Functions - UDF), belirli parametreler alarak bir işlem gerçekleştiren ve geriye bir değer döndüren, yeniden kullanılabilir SQL kod bloklarıdır. SELECT sorguları içinde doğrudan kullanılabilmeleri en büyük avantajlarıdır.

4.1. Fonksiyon Türleri

1. **Geriye Sabit Değer Döndüren (Scalar Functions):** Tek bir skaler değer (örn: int , varchar , datetime) döndürürler. Matematiksel hesaplamalar, metin birleştirme gibi işlemler için idealdirler.

2. **Geriye Sorgu Döndüren (Inline Table-Valued Functions):** Geriye tek bir SELECT ifadesinin sonucunu TABLE olarak döndürürler. Parametrelili VIEW 'ler gibi düşünülebilir.

3. **Geriye Tablo Değişkeni Döndüren (Multi-statement Table-Valued Functions):** Karmaşık işlemler yaparak geriye bir tablo değişkeni döndürürler. BEGIN...END bloğu içinde birden fazla SQL ifadesi barındırabilirler.

4.2. Stored Procedure ve Function Arasındaki Farklar

Aşağıdaki tablo, iki yapı arasındaki temel farkları özetlemektedir:

Özellik	Stored Procedure	User-Defined Function
Geri Dönüş Değeri	Opsiyoneldir; değer döndürmeyebilir.	Zorunludur; her zaman bir değer (skaler veya tablo) döndürür.
DML İşlemleri	INSERT , UPDATE , DELETE kullanabilir.	Sadece SELECT kullanabilir; DML komutlarına izin verilmez.
Parametreler	Giriş (INPUT) ve çıkış (OUTPUT) parametreleri alabilir.	Sadece giriş (INPUT) parametreleri alabilir.
Hata Yönetimi	TRY...CATCH bloklarını kullanabilir.	TRY...CATCH bloklarını kullanamaz.
Transaction Yönetimi	TRANSACTION işlemlerini destekler.	TRANSACTION yapısını desteklemez.
Çağırma Şekli	EXECUTE veya EXEC komutu ile çağrılır.	SELECT sorgusu içinde veya FROM yan tümcesinde çağrılır.
Çağrılabilirlik	Bir Function 'ı çağırabilir.	Bir Stored Procedure 'ü çağıramaz.

Bölüm 5: Saklı Yordamlar (Stored Procedure Kullanımı)

Saklı Yordamlar (Stored Procedure - SP), veritabanı sunucusunda saklanan ve ilk derlemeden sonra tekrar derlenmeye ihtiyaç duymayan, önceden derlenmiş SQL ifadeleridir. Bu özellikleri sayesinde yüksek performans sunarlar.

5.1. Stored Procedure'lerin Avantajları

- **Performans:** İlk çalıştırmada derlenir ve sonraki çalıştırmalarda bu derlenmiş plan kullanılır, bu da tekrar eden işlemler için hızı artırır.
- **Güvenlik:** Kullanıcılara doğrudan tablo erişimi vermek yerine, belirli işlemleri yapan prosedürlere erişim yetkisi verilebilir. Bu, SQL Injection gibi saldırılara karşı koruma sağlar.
- **Ağ Trafığının Azalması:** Uygulama tarafından sunucuya uzun SQL sorguları göndermek yerine, sadece prosedürün adı ve parametreleri gönderilir.
- **Modülerlik:** Karmaşık iş mantığı, yeniden kullanılabilir ve yönetilebilir prosedürler halinde düzenlenebilir.

5.2. Stored Procedure Yönetimi

Parametresiz SP Oluşturma

```
CREATE PROC TumBilgiler
AS
SELECT * FROM TblOgrenci;
```

Çalıştırmak için: EXEC TumBilgiler;

Parametrelili SP Oluşturma

Prosedürler, dışarıdan değer alarak dinamik sorgular oluşturabilir.

```
CREATE PROC OgrenciBilgiGetir @OgrenciAdi varchar(40)
AS
SELECT * FROM TblOgrenci WHERE Ad = @OgrenciAdi;
```

Çalıştırmak için: EXEC OgrenciBilgiGetir 'Ayşe';

DML İşlemleri İçin SP Kullanımı

INSERT , UPDATE , DELETE işlemleri için de prosedürler oluşturulabilir.

```
CREATE PROC OgrenciEkle
    @OgrenciAd varchar(30),
    @OgrenciSoyad varchar(30),
    @Sehir int,
    @Cinsiyet int
AS
INSERT INTO TblOgrenci VALUES (@OgrenciAd, @OgrenciSoyad, @Sehir, @Cinsiyet);
```

Çalıştırmak için: EXEC OgrenciEkle 'Gökhan', 'ARMAN', 1, 2;

- **Güncelleme:** ALTER PROC <Prosedür_Adi> ...

- **Silme:** DROP PROC <Prosedür_Adi>

Bölüm 6: Otomatik Tetikleyiciler (TRIGGER Kullanımı)

Trigger (Tetikleyici), bir tabloda belirli bir DML olayı (INSERT , UPDATE , DELETE) gerçekleştiğinde otomatik olarak çalışan özel bir Stored Procedure türüdür. Genellikle veri bütünlüğünü korumak, karmaşık iş kurallarını uygulamak ve denetim (auditing) amacıyla kullanılırlar.

6.1. Trigger Türleri

1. **AFTER (veya FOR) Trigger:** İlişkili DML işlemi (ekleme, silme, güncelleme) tamamlandıktan *hemen sonra* tetiklenir. En yaygın kullanılan türdür.

2. **INSTEAD OF Trigger:** İlişkili DML işlemi *yerine* tetiklenir. Yani, asıl INSERT , UPDATE veya DELETE işlemi gerçekleşmez; onun yerine trigger içindeki kodlar çalışır. Genellikle VIEW 'leri güncellenebilir hale getirmek veya karmaşık doğrulama işlemleri yapmak için kullanılır.

6.2. inserted ve deleted Sanal Tabloları

Trigger'lar, işlem sırasında etkilenen verileri tutan iki özel sanal tabloya erişebilir:

- **inserted:** INSERT işlemiyle eklenen yeni satırları veya UPDATE işlemiyle güncellenen satırların yeni hallerini içerir.

- **deleted:** DELETE işlemiyle silinen satırları veya UPDATE işlemiyle güncellenen satırların eski hallerini içerir.

Bu tablolar, trigger içindeki mantığı kurmak için kritik öneme sahiptir.

6.3. Trigger Oluşturma ve Yönetimi

Genel Sözdizimi:

```
CREATE TRIGGER trigger_adi
ON tablo_adi
{FOR | AFTER | INSTEAD OF} {INSERT, UPDATE, DELETE}
AS
BEGIN
    -- SQL ifadeleri
END
```

- **Örnek (Loglama):** Silinen öğrenci kayıtlarını başka bir tabloya kaydetmek.

6.4. İşlem Kontrolü ve Hata Yönetimi

- **ROLLBACK TRANSACTION:** Trigger içinde bu komut kullanıldığında, tetikleyiciyi başlatan DML işlemi tamamen geri alır. Bu, bir iş kuralı ihlal edildiğinde işlemin iptal edilmesi için kullanılır.

- **RAISERROR:** Kullanıcıya özel bir hata mesajı göstermek için kullanılır. Genellikle ROLLBACK ile birlikte kullanılarak işlemin neden başarısız olduğu hakkında bilgi verilir.

- **Örnek:** Belirli bir sınıfa erkek öğrenci kaydedilmesini engellemek.

6.5. Trigger'ları Aktifleştirme ve Pasifleştirme

Oluşturulan trigger'lar varsayılan olarak aktiftir. Gerektiğinde geçici olarak devre dışı bırakılabilirler.

- **Pasifleştirme:** `DISABLE TRIGGER trigger_adi ON tablo_adi;`
- **Aktifleştirme:** `ENABLE TRIGGER trigger_adi ON tablo_adi;`
- **Tüm Trigger'lar:** Bir tablodaki tüm trigger'ları pasifleştirmek veya aktifleştirmek için `trigger_adi` yerine `ALL` kullanılabilir.